

Búsqueda en Espacios de Estados

Unidad 2 — Algoritmos de búsqueda no informada, informada y adversarial.

Contenido

01 Representación formal de un espacio de estados

02 Búsqueda no informada: BFS y DFS

03 Búsqueda informada: A* y Búsqueda Voraz

04 Comparación de algoritmos

05 Búsqueda adversarial: Minimax (Tres en Raya)

01

**Representación Formal de
un Espacio de Estados**

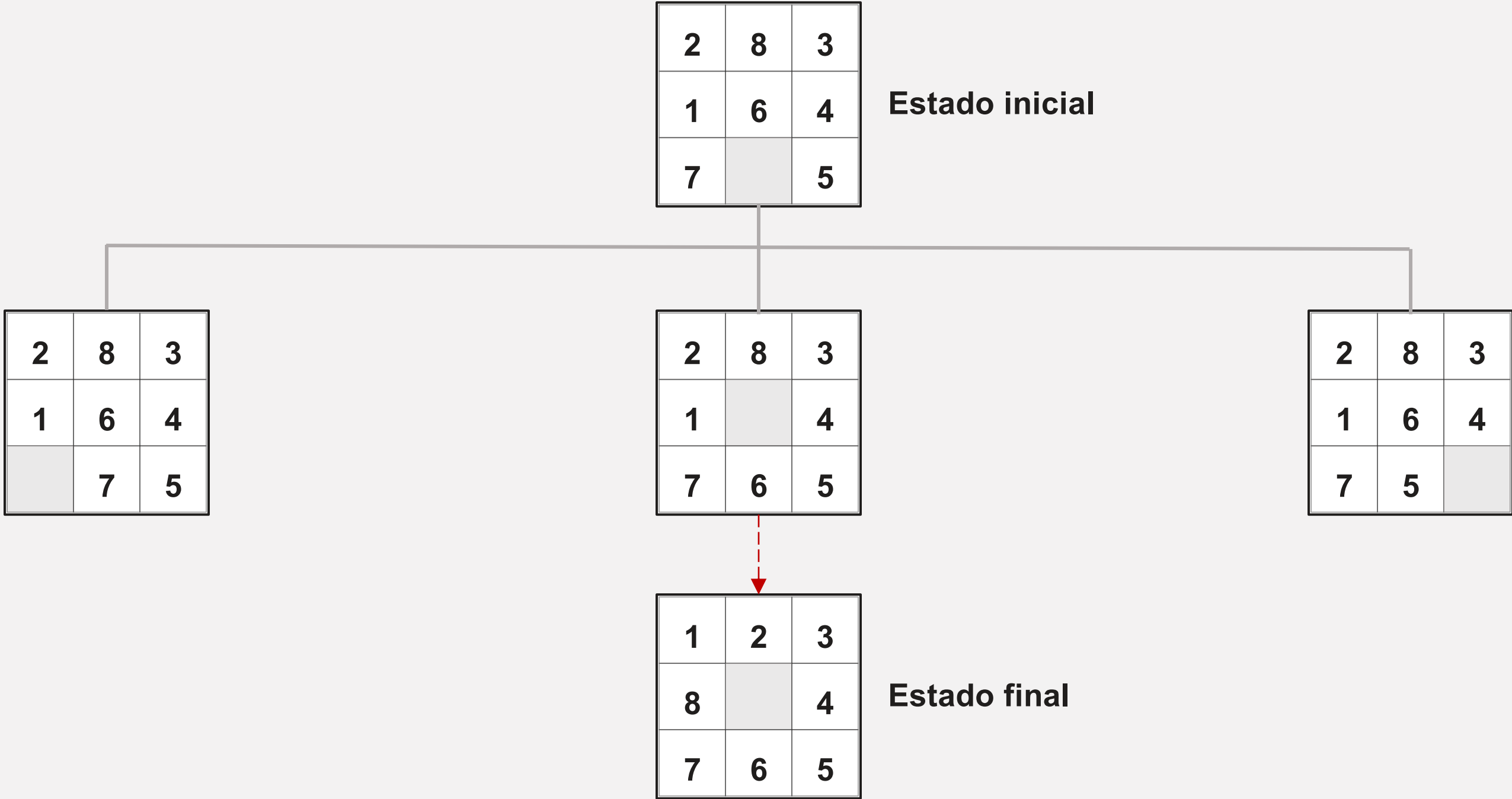
Un espacio de estados como grafo dirigido

- Los nodos representan los estados
- Las aristas representan las acciones que permiten pasar de un estado a otro

Notación	Significado
$S = \{s_0, s_1, ..., s_n\}$	Conjunto de estados
$O = \{a_1, a_2, ..., a_m\}$	Conjunto de operadores (acciones)
$f(s,a) \rightarrow s'$	Función de transición: aplicar a en s produce s'
$S_goal \subset S$	Estado(s) meta

El 8-puzzle es el problema usado en todo este notebook para poner en práctica esta representación.

El 8-puzzle como espacio de estados



02

**Búsqueda No Informada:
BFS y DFS**

¿Qué es la búsqueda no informada?

- Explora el espacio de estados sin usar información adicional sobre qué tan cerca está la solución
- Solo conoce: el estado inicial, el estado meta, y qué transiciones son posibles
- BFS (Breadth-First Search): explora por niveles, en todas las direcciones antes de profundizar — usa una cola (FIFO)
- DFS (Depth-First Search): explora un camino hasta el final antes de retroceder — usa una pila (LIFO)

BFS resuelve el 8-puzzle

- estado_inicial = "103425786"
- Resultado real: solución en 3 pasos
- BFS explora nivel por nivel, así que la primera solución que encuentra ya es la más corta posible

```
def bfs_8_puzzle (inicial, meta):  
    cola = deque([inicial])  
    predecesor = {inicial: None }  
  
    while cola: actual = cola.popleft()  
        # FIFO  
        for sucesor in generar_sucesores(actual):  
            if sucesor not in predecesor:  
                predecesor[sucesor] = actual  
                cola.append(sucesor)  
                if sucesor == meta:  
                    return reconstruir_camino(predecesor, meta)
```

DFS resuelve el mismo puzzle

- Mismo estado inicial y meta que con BFS
- DFS solo cambia FIFO por LIFO — pero encuentra un camino mucho más largo que el óptimo

```
def dfs_8_puzzle (inicial, meta):  
    pila = [inicial]  
    predecesor = {inicial: None }  
  
    while pila: actual = pila.pop()  
        # LIFO  
        for sucesor in generar_sucesores(actual):  
            if sucesor not in predecesor:  
                predecesor[sucesor] = actual  
                pila.append(sucesor)  
                if sucesor == meta:  
                    return reconstruir_camino(predecesor, meta)
```

Mismo estado, mismo meta — ¿cuántos pasos?

3

pasos — BFS (óptimo)

11

pasos — DFS

DFS encuentra una solución válida, pero al no explorar por niveles no hay garantía de que sea la más corta.

03

**Búsqueda Informada:
A* y Búsqueda Voraz**

Heurísticas y la función $f(n) = g(n) + h(n)$

$$f(n) = g(n) + h(n)$$

$g(n)$: costo real recorrido · $h(n)$: estimación heurística hasta la meta

- Una heurística estima qué tan cerca está un estado de la meta, sin evaluar exhaustivamente todas las posibilidades
- Heurística de Manhattan para el 8-puzzle: suma de las distancias de cada ficha a su posición objetivo
- heapq implementa la frontera como cola de prioridad: siempre extrae el estado con menor $f(n)$

A* resuelve el mismo puzzle: 3 pasos

- Sobre el mismo estado inicial de BFS/DFS, A* también encuentra la solución óptima: 3 pasos
- A diferencia de BFS, no explora todo un nivel completo — la heurística lo guía directamente hacia estados prometedores

```
def heuristica_manhattan (estado, meta):  
    distancia = 0  
  
    for i, ficha in enumerate (estado):  
        if ficha != "0" :  
            pos_meta = meta.index(ficha)  
            fila_a, col_a = divmod (i, 3)  
            fila_m, col_m = divmod (pos_meta, 3)  
            distancia += abs(fila_a-fila_m) + abs (col_a-col_m)  
  
    return distancia # f(n) = g(n) + h(n)  
# heapq mantiene el estado de menor f(n) al frente
```

Búsqueda Voraz: solo $h(n)$

- Función de evaluación: $f(n) = h(n)$ — ignora por completo el costo ya recorrido $g(n)$
- En cada paso elige el estado que "parece" más cercano a la meta, sin importar cuánto costó llegar hasta ahí
- Puede expandir menos nodos que A^* cuando la heurística es muy precisa
- No garantiza una solución óptima: puede comprometerse con un camino que resulte más largo

A* vs. Voraz, en un caso donde sí se nota

8

10

pasos — A* (óptimo)

pasos — Voraz (válida, no óptima)

- Al guiarse solo por la heurística, la Búsqueda Voraz avanzó por un camino que "parecía" prometedor, sin considerar la distancia ya recorrida
- A* combina $g(n)$ y $h(n)$, por eso encuentra el camino verdaderamente más corto

04

Comparación de Algoritmos

BFS, DFS, Voraz y A* — resumen

Algoritmo	Frontera	Usa heurística	Óptimo	Memoria
BFS	FIFO (cola)	No	Sí	Alta
DFS	LIFO (pila)	No	No	Baja
Voraz	Prioridad, $f(n)=h(n)$	Sí	No	Moderada
A*	Prioridad, $f(n)=g(n)+h(n)$	Sí	Sí (heurística admisible)	Moderada

- BFS garantiza el óptimo, pero su costo de memoria lo vuelve poco práctico en problemas grandes
- A* logra el mismo óptimo que BFS, guiado por la heurística, de forma mucho más eficiente

05

**Búsqueda Adversarial:
Minimax (Tres en Raya)**

El algoritmo Minimax

- A diferencia de BFS/DFS/A*, aquí hay un oponente que también elige sus movimientos — no basta con buscar el mejor camino propio
- MAX (la IA): elige el movimiento que maximiza su puntaje · MIN (el oponente): elige el movimiento que lo minimiza
- Puntaje terminal: +10 menos la profundidad si gana la IA, −10 más la profundidad si gana el humano, 0 si es empate
- Poda alfa-beta: descarta ramas que ya no pueden cambiar la decisión final, sin explorarlas — mismo resultado, menos trabajo

La aplicación: minimax-tictactoe/

- Interfaz interactiva en Streamlit para jugar Tres en Raya contra la IA, con tres niveles de dificultad
- Aleatoria total: la IA elige un movimiento al azar · Fácil (subóptima): entre los movimientos razonablemente buenos
- Perfecta (MinMax completo): nunca pierde — juega el movimiento óptimo exacto en cada turno
- La aplicación también visualiza el árbol de búsqueda y qué ramas fueron podadas por alfa-beta

```
cd Unidad 2/minimax-tictactoe && streamlit run app.py
```

Resumen de la Unidad 2

Tema	Recurso del curso
Representación de estados, BFS, DFS, A*, Voraz	Unidad 2/1_buscar-estados-ia.ipynb
Búsqueda adversarial: Minimax + poda alfa-beta	Unidad 2/minimax-tictactoe/ (app Streamlit)

BFS y A* garantizan la solución óptima; A* lo hace de forma mucho más eficiente gracias a la heurística

DFS y la Búsqueda Voraz sacrifican la garantía de optimalidad a cambio de menor costo de memoria o de cómputo

¡Gracias!

Estos algoritmos de búsqueda son la base conceptual para todo el razonamiento por pasos que viene después en el curso.